

1. Verzweigungen (Fortsetzung)
2. Schleifen
3. Arrays
4. Methoden
5. Ausnahmebehandlung

Verzweigungen (Fortsetzung)

- Werden benutzt wenn Anweisungen nur unter einer Bedingung ausgeführt werden sollen
- Verzweigungen können sich beliebig oft weiter verzweigen
- Verzweigungstypen
 - if (else)
 - switch

- Eine Anweisung wird ausgeführt, wenn eine Bedingung erfüllt ist
- Erfüllt heißt „true“ in einem boolean oder durch Vergleichsoperatoren
- Die Bedingung lässt sich mit logischen Operatoren zusammensetzen

■ Syntax:

```
if (<Bedingung>
{
    <Anweisung>;
}
```

■ Beispiel:

```
int z1 = 2, z2 = 5;
if (z1 < z2)
{
    System.out.print(z1 +
        " ist kleiner als " + z2);
}
```

- Eine if-Bedingung kann mit „else“ erweitert werden
- Der Code von else wird ausgeführt, wenn die if-Bedingung nicht erfüllt wurde
- durch „else if“ kann noch weiter verzweigt werden

■ Syntax:

```
■ if (<Bedingung>
{
    <Anweisung>;
}
else if (<Bedingung>)
{
    <Anweisung>;
}
else
{
    <Anweisung>;
}
```

- Es wird auf Gleichheit einer Variable geprüft
- Werten der Variable wird die Anweisung entgegengesetzt
- Das Ende jeder Anweisung wird mit „break“ beendet
- Mit „default“ wird die Anweisung bezeichnet die sonst ausgeführt wird

■ Syntax:

```
■ switch (<variable>
{
    case <Konstante 1>:
        <Anweisung>;
        break;

    ...
    default:
        <Anweisung>;
}
```

Switch – Bedingung - Beispiel



- Einem int „dayOfWeek“ soll je nach Wochentag eine Zahl eins bis sieben zugeordnet worden sein. Der Wochentag soll in die Konsole geschrieben werden

Switch – Bedingung - Beispiel



- Einem int „dayOfWeek“ soll je nach Wochentag eine Zahl eins bis sieben zugeordnet worden sein. Der Wochentag soll in die Console geschrieben werden

- **Beispiel:**

```
■ switch (dayOfWeek)
{
    case 1:
        System.out.println("Montag");
        break;
    case 2:
        System.out.println("Dienstag");
        break;
    case 3:
        System.out.println("Mittwoch");
        break;
    case 4:
        System.out.println("Donnerstag");
        break;
    case 5:
        System.out.println("Freitag");
        break;
    case 6:
        System.out.println("Samstag");
        break;
    case 7:
        System.out.println("Sonntag");
        break;
    default:
        System.out.println("Unbekannt");
}
```

Schleifen

- Schleifen wiederholen Code solange bis eine Bedingung erfüllt ist
- Schleifentypen
 - while – Schleife (kopfgesteuert)
 - do-while – Schleife (fußgesteuert)
 - for – Schleife (kopfgesteuert)

- Ähnelt von der Syntax der if-Bedingung
- Die while-Schleife wiederholt die Anweisung, solange die Bedingung erfüllt ist



Wenn sich der Zustand der Bedingung nie ändert wird die Schleife nie beendet! (z.B. while (true))

■ Syntax:

```
■ while (<Bedingung>)  
{  
    <Anweisung>;  
}
```

■ Beispiel:

```
■ int x=0;  
  while (x<=5)  
  {  
      x++;  
  }
```

- Die Fakultät einer Zahl soll mit Hilfe einer while-Schleife berechnet werden. Dabei soll das int n bereits mit der Zahl 5 initialisiert worden sein
- **Tipp zur Berechnung:**
 - Die Fakultät einer Zahl $n! = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 2 \cdot 1$
 - Beispiel: $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$

- Die Fakultät einer Zahl soll mit Hilfe einer while-Schleife berechnet werden. Dabei soll das int n bereits mit der Zahl 5 initialisiert worden sein

- **Beispiel:**

- //das Ergebnis
int fak=1;
//die Laufvariable
int i=1;
while (i<=n)
{
 fak=fak*i;
 i++;
}

- Funktioniert so ähnlich wie die while-Schleife
- Wird jedoch mindestens einmal ausgeführt, da die Überprüfung der Bedingung am Ende statt findet

- **Syntax:**

```
■ do
{
    <Anweisung>;
}
while (<Bedingung>)
```

■ Syntax:

```
■ for (<Variable> = <Startwert>; <Bedingung>; <Variable> = <Variable> + <Schrittweite>)  
{  
    <Anweisung>;  
}
```

- Als Variable wird in der Regel „i“ verwendet
- Die Schleife läuft, bis die Bedingung erfüllt ist
- Bei jedem Durchgang verändert sich die Variable um die angegebene Einheit

- Was wird in dem Beispiel in der Konsole ausgegeben?

- **Beispiel:**

- ```
for (int i=1; i<15; i++)
{
 for (int j=1; j<=i; j++)
 {
 System.out.print("_");
 }
 System.out.println();
}
```

# For - Schleifen - Beispiel



- Was wird in dem Beispiel in der Konsole ausgegeben?

A series of 12 horizontal black lines of increasing length, arranged vertically. The lines start from a common left edge and extend to the right, with each subsequent line being longer than the one above it. The lines are evenly spaced, creating a staircase-like effect.

## ■ Beispiel:

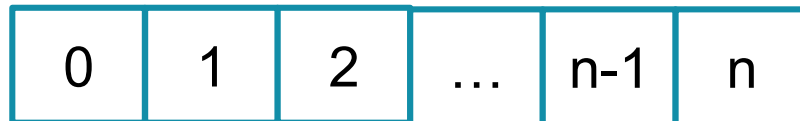
- ```

■ for (int i=1; i<15; i++)
{
    for (int j=1; j<=i; j++)
    {
        System.out.print("_");
    }
    System.out.println();
}

```

Eindimensionale Arrays

- In einem Array können endlich viele Werte eines Datentyps gespeichert werden
- Zwei Arten der Initialisierung
 - Initialisierung mit `new`
 - Direkte Initialisierung
- Zugriff über den Index, angefangen bei 0



Initialisierung mit new

- **Syntax:**

- `<Datentyp>[ ] <Name> = new <Datentyp>[<Anzahl Stellen>];`

- **Beispiel:**

- `int[ ] zahlenArray = new int[10];`

- Die eckige Klammer beinhaltet die Anzahl verfügbarer Stellen im Array
- Die Elemente werden mit null oder 0 initialisiert

Direkte Initialisierung

- **Syntax:**

- `<Datentyp>[ ] <Name> = {<Wert 1>, <Wert 2>, ..., <Wert n>;}`

- **Beispiele:**

- `int[ ] zahlenArray = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};`

- `char[ ] halloArray = {'H', 'a', 'l', 'l', 'o'};`

- Ein Array kann auch durch explizite Angabe von Werten erzeugt werden
- Die Werte werden in geschweifte Klammern geschrieben und mit Komma getrennt

- Der Zugriff erfolgt über den Namen des Arrays und dem Index des Elements
- Der Zugriff auf das Element wird unterschieden in
 - lesender Zugriff
 - schreibender Zugriff
- Der Zugriff auf das Array wird unterschieden in
 - direkt
 - sequentiell

Lesender Zugriff (Direkt)

- **Syntax:**

- `<Name Array> [Index];`
- `[Datentyp] <Name Variable> = <Name Array>[Index]`

- **Beispiele:**

- `int ersteZahl = zahlenArray[0];`
- `System.out.println("Zweite Zahl: " + zahlenArray[1]);`

Schreibender Zugriff (Direkt)

- **Syntax:**

- `<Name Array> [Index] = <Wert>;`

- **Beispiele:**

- `zahlenArray[2] = 2;`

- `halloArray[1] = 'a';`

Sequentieller Zugriff

- Ein Array kann mit einer Schleife durchlaufen werden
- Bei jedem Schleifendurchlauf kann auf den Wert eines Indexes zugegriffen werden

■ Beispiel (schreibend):

```
■ for (int i=0;i<=9;i++)  
  {  
      zahlenArray[i]=i;  
  }
```

■ Beispiel (lesend):

```
■ for (int i=0;i<=9;i++)  
  {  
      System.out.println(zahlenArray[i]);  
  }
```

Eindimensionale Arrays - Length

- Die Länge des Arrays erhält man mit dem Befehl "length"

- **Syntax:**
 - <Name Array>.length

- **Beispiel:**

- for (int i=0; i ? halloArray.length; i++)
{
 System.out.print(halloArray[i]);
}

Eindimensionale Arrays - Beispiel

- Welcher Vergleichsoperator muss in `?` ?

- **Beispiel:**

- ```
for (int i=0; i ? halloArray.length; i++)
{
 System.out.print(halloArray[i]);
}
```

- Welcher Vergleichsoperator muss in ? ?

<= ■ `ArrayIndexOutOfBoundsException`

>= ■ geht nicht rein

== ■ geht nicht rein

> ■ geht nicht rein

< ■ 

- **Beispiel:**

- ```
for (int i=0; i ? halloArray.length; i++)  
{  
    System.out.print(halloArray[i]);  
}
```

- Es geht auch einfacher...

- **Beispiel:**

- `int[] arrName = new int[5];`
 - `for (int i: arrName)`
 - `{`
 - `System.out.print(i);`
 - `}`

Mehrdimensionale Arrays

Mehrdimensionale Arrays

- Ist analog zum eindimensionalen Array aufgebaut
- Wird um die weitere Dimension ergänzt

	0	1	2	...	n-1	n
0						
1						
2						
...						
m-1						
m						

Initialisierung mit new

- **Syntax:**

- `<Datentyp>[][] <Name> = new <Datentyp>[<Stellen>][<Stellen>;`

- **Beispiel:**

- `int[][] zahlenArray = new int[10][2];`

- Die weitere Dimension wird in der zweiten Klammer dargestellt

Direkte Initialisierung

■ Syntax:

```
■ <Datentyp>[][] <Name> = {{<Wert 1|1>, <Wert 1|2>, ..., <Wert 1|n>},  
                           {<Wert 2|1>, <Wert 2|2>, ..., <Wert 2|n>},  
                           {  
                               ....  
                           },  
                           {<Wert m|1>, <Wert m|2>, ..., <Wert m|n>}};
```

■ Beispiele:

```
■ int[][] zahlenArray = {{0, 1, 2, 3, 4, 5, 6, 7, 8, 9},  
                         {10,11,12,13,14,15,16,17,18,19}};
```

- Jede Zeile wird in geschweifte Klammern geschrieben
- Die Klammern jeder Zeile werden durch Komma getrennt

Lesender Zugriff (Direkt)

- **Syntax:**

- `<Name Array> [Index][Index];`
- `[Datentyp] <Name Variable> = <Name Array>[Index][Index]`

- **Beispiele:**

- `int ersteZahl = zahlenArray[0][0];`
- `System.out.println("Zweite Zahl, dritte Zeile: " + zahlenArray[1][2]);`

Schreibender Zugriff (Direkt)

- **Syntax:**

- `<Name Array> [Index][Index] = <Wert>;`

- **Beispiele:**

- `zahlenArray[2][0] = 2;`

Sequentieller Zugriff

- Für die weitere Dimension kommt eine weitere Schleife dazu

■ Beispiel (schreibend):

```
■ for (int i=0;i<=9;i++)
{
    for (int j=0;j<=i;j++)
    {
        zahlenArray[i][j]=i+(j*10);
    }
}
```

■ Beispiel (lesend):

```
■ for (int j = 0; j <= 1; j++)
{
    for (int i = 0; i <= 9; i++)
    {
        System.out.print(zahlenArray[i][j]+" ");
    }
    System.out.println(); //Leerzeile
}
```

- Die Länge der Tiefe wird über einen kleinen Umweg ermittelt

- **Beispiel:**

- ```
for (int j = 0; j < zahlenArray[0].length; j++)
{
 for (int i = 0; i < zahlenArray.length; i++)
 {
 System.out.print(zahlenArray[i][j]+" ");
 }
 System.out.println();
}
```

- **Syntax:**

- `<Name Array>[0].length`

# Methoden

- Funktionen können in Methoden ausgelagert werden
- Eine Methode besteht aus
  - Modifizierer
    - z.B. public, private
  - Rückgabetyp
    - void, int, String etc.
    - Die Rückgabe erfolgt über „return“
  - Methodenname
    - It. Konvention beginnend mit einem kleinen Buchstaben
    - sollen die Funktion der Methode beschreiben
  - übergebene Parameter

## ■ Syntax:

- <Modifizierer> void <Name> (<Parameter>)
- <Modifizierer> <Datentyp> <Name> (<Parameter>)

## ■ Beispiele

- ```
private int add (int a, int b)
{
    return a+b;
}
```
- ```
public void multiply (int a, int b)
{
 int result = a*b;
 System.out.println(result);
}
```

- ```
int num1 = 2;
int num2 = 4;
//Addition der Zahlen
num1 = add(num1, num2);
//Multiplikation der Zahlen
multiply(num1, num2);
```

- Welche Rückgabetypen benötigen die folgenden Methoden?

- `public ? add(int num1, int num2)`
`{`
 `int sum = num1+num2;`
 `return sum;`
`}`
- `public ? printNumber(int num)`
`{`
 `System.out.println("Ausgabe: "+num);`
`}`
- `public ? subtract(long num1, long num2)`
`{`
 `return num1-num2;`
`}`

- Welche Rückgabetypen benötigen die folgenden Methoden?

- `public ? add(int num1, int num2)`
{
 int sum = num1+num2;
 return sum;
}

int

- `public ? printNumber(int num)`
{
 System.out.println("Ausgabe: "+num);
}

void

- `public ? subtract(long num1, long num2)`
{
 return num1-num2;
}

long

Ausnahmebehandlung

Exceptions (Ausnahmen)

- Fehler die während der Programmausführung auftreten können
- Solche Exceptions können sein
 - `NumberFormatException`
 - `NullPointerException`
 - `IndexOutOfBoundsException`
- Können behandelt werden

- Besteht aus einem try-, einem catch und einem finally-Block
- Try
 - Beinhaltet den Code, der zu einer Exception führen kann
 - Wenn eine Exception auftritt, wird sofort zum catch-Block gesprungen
- Catch
 - Beinhaltet den Code, der im Falle einer Exception ausgeführt werden soll
- Finally
 - Optional
 - Beinhaltet Code der anschließend in jedem Fall ausgeführt werden soll

Ausnahmebehandlung mit try-catch

■ Syntax:

```
■ try
{
    <Anweisung>;
}
catch(Exception e)
{
    <Anweisung>;
}
```

```
■ try
{
    <Anweisung>;
}
catch(<Exception1> e)
{
    <Anweisung>;
}
catch(<Exception2> e)
{
    <Anweisung>;
}
```

```
■ try
{
    <Anweisung>;
}
catch(Exception e)
{
    <Anweisung>;
}
finally
{
    <Anweisung>;
}
```

Ausnahmebehandlung mit try-catch - Beispiel



- Die Methode `private int divide(int num1, int num2)` kann eine `ArithmeticException` werfen. Die `int x1, x2` sind bereits initialisiert worden. Das Ergebnis soll „-1“ gesetzt werden, wenn die Exception aufgetreten ist. In jedem Fall soll anschließend das Ergebnis in der Konsole ausgegeben werden

Ausnahmebehandlung mit try-catch - Beispiel

- Die Methode `private int divide(int num1, int num2)` kann eine `ArithmeticException` werfen. Die `int x1, x2` sind bereits initialisiert worden. Das Ergebnis soll „-1“ gesetzt werden, wenn die Exception aufgetreten ist. In jedem Fall soll anschließend das Ergebnis in der Konsole ausgegeben werden

- **Beispiel:**

```
int result;  
try  
{  
    result = divide(x1, x2);  
}  
catch(ArithmeticException e)  
{  
    result = -1;  
}  
finally  
{  
    System.out.println( "Das Ergebnis ist "+result);  
}
```

Ausnahmen weitergeben mit throws

- Kann einer Methode angehängt werden
- Dadurch wird die Exception an den Aufrufer zurück gegeben

- **Syntax:**

- <Methode> throws <Exception>

- **Beispiel:**

- `private int parseStringToInt(String string) throws NumberFormatException`

- Es ist möglich selbst Exceptions auszulösen
- Der Programmablauf wird an dieser Stelle abgebrochen
 - z.B. in Methoden sinnvoll

- **Syntax:**

- throw new <Name der Exception>();

- **Beispiel:**

- private void checkAge(int age)
 {
 if (age<0)
 throw new IllegalArgumentException("Alter unter 0");
 ...
 }